Started on	Wednesday, 3 June 2020, 12:50 PM
State	Finished
Completed on	Wednesday, 3 June 2020, 1:25 PM
Time taken	35 mins 1 sec
Marks	6.00/6.00
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Complete

Mark 1.00 out of 1.00

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mari numere. Secvența este prea mare pentru a fi stocată integral in memorie.

Ma gandesc sa folosesc o coada de prioritati minima, in care voi baga primele K numere. Apoi de la numarul K + 1 pana la N o sa compar minimul din coada cu elementul curent. Daca minimul din coada este mai mare decat numarul curent, scot minimul din coada si adaug noul numar in coada.

De exemplu, pt sirul 1,2,3,0,-1,5 si k = 2 adaug 1,2 in coada dupa la 3 inlocuiesc minimul cu 3, adica voi avea 2 3 in coada, la 0 dau skip, la -1, skip la 5, inlocuiesc 5 cu 2 in coada si obtin 3 5.

Algoritm de tip greedy

n log k- folosim un minheap

n * k - 0.5 din punctaj

Comment:

Question **2**

Complete

Mark 1.00 out of 1.00

Se dă un vector *v* cu *n* intervale de forma [a_i, b_i]. Se cere numărul maxim de intervale disjuncte (care nu se intersectează cu niciun interval). Descrieți un algoritm eficient.

Abordare de tip greedy

Sortez screscator intervalele [ai, bi] dupa timpul de finalizare in n log n. Iau un contor de intervale disjuncte, initializat la 0. Dupa, imi iau start stop la timpul la primul element din vectorul sortat [a0, b0]. Apoi, parcurg vectorul de intervale si daca intervalul curent este continut in intervalul [start, stop] inseamna ca el este redundant si poate fi omis. Altfel, intervalul intervalul curent nu se intersecteaza cu cel anterior si ar trebui sa cresc contor-ul de intervale disjuncte.

Greedy: se sortează apoi se parcurg intervalele de la stânga la dreapta. O(n log n).

Comment:

Question **3**

Complete

Mark 1.00 out of 1.00

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *n* noduri care au un ciclu de lungime cel puțin 2.

Cred ca as putea folosi o abordare de tip backtracking in care voi genera toate grafurile posibile si asta imi va lua cam O(2^(n^2)) (ma uit in matricea de adiacenta si iau cele n^2 elemente si le fac pe rand 0 sau 1, mai putin pe cele de pe diagonala principala, care vor ramane 0). Dupa, verific in prima faza daca elementul a[i][j] = a[j][i] in matrice, asta insemnand ca muchia este bidirectionala. Daca nu, aplic un kosaraju/tarjan si pot sa iau toate componentele conexe, dupa care verific daca am mai putin de n componente.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunț.

Comment:

Question **4**

Complete

Mark 1.00 out of 1.00

Cum puteți **să calculați** numărul total de drumuri distincte între 2 noduri dintr-un graf **orientat aciclic**?

Ma gandesc ca as putea folosi o sortare topologica a nodurilor, fiind DAG. Dupa as aplica o tehnica de programare dinamica, unde as avea un dp[i] = numarul de noduri prin care se poate ajunge din i la destinatie. Ma gandesc, ca as avea doua for-uri (pentru noduri din sortare, pentru vecinii nodului curent), asa ca ma astept la o complexitate n^2. Mai trebuie sa stabilesc dinamica exacta a problemei.

Sortare topologica apoi dinamică pentru a calcula numarul de drumuri pana la un anumit nod.

Comment:

Question **5**

Complete

Mark 1.00 out of 1.00

Se dă un graf orientat cu *x* noduri, având capacitați numere din șirul Fibonacci. Se știe că există *A* surse și *B* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la toate destinațiile.

Ma gandesc sa unesc cele A surse si cele B drene intr-un nod sursa si un nod drena. Dupa voi putea aplica algoritmul Edmonds-Karp pentru a gasi cantitatea maxima de flux dintre sursa si destinatie.

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comment:

Question **6**

Complete

Mark 1.00 out of 1.00

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Cred ca voi putea aplica algoritmul lui Tarjan pe aceasta problema, pentru a determina componentele biconexe ale fortaretei. Acest lucru se datoreaza faptului ca daca intr-o componenta biconexa, un nod este eliminat, atunci subgraful asociat componentei biconexe ramane conex. Astfel, daca o fortareata e bombardata, celelalte vor comunica in continuare. Tarjan presupune doar o parcurgere dfs, asa ca ma astept la o complexitate de O(N + M). Daca am fortarete izolate, cred ca nu le voi considera componente biconexe separate, intrucat ele nu comunica cu nimeni, dar nici pe cele cu 2 noduri, intrucat acestea vor deveni izolate. Daca am ceva similar cu un lant, atunci nu cred ca e problema, fiecare mai putin capatul stanga si dreapta fac parte dintr-o comp biconex separaa

Gasim toate componentele biconexe.

Comment:

◄ Examen PA - Subiect II - 03.06.2020

Jump to...

Examen Subiectul 1 - Late submit ►